Beyond Chatbots

Building Generative UI with Flutter GenUI SDK

Explore how Flutter GenUI flips the script: UI that adapts to user intent rather than users adapting to static interfaces.

March 28, 2026 • Namma Flutter, Coimbatore

The UX Gap

Why Static Apps and Chatbots Still Miss User Intent

Same Screen for Everyone

Most apps present identical layouts to every user, regardless of context, urgency, or skill level. When the UI is generic, users must do the adaptation work instead of the product adapting to them.

Discovery Overload

Users are forced through tabs, filters, and long scrolls just to find one relevant action. Too many widgets and too many paths increase friction and reduce confidence.

Intent Mismatch

Users usually know what they want, but apps still make them translate intent into navigation steps. This mismatch creates unnecessary clicks, drag, and repeated back-and-forth.

Chatbot Gap

Chat interfaces can answer questions, but answers alone are often not actionable in-app. Users still need structured UI controls to compare options, decide quickly, and complete tasks.

What if apps understood intent first, then rendered the right UI instantly? That is GenUI: Widgets + AI.

March 28, 2026 • Namma Flutter, Coimbatore

STATIC UI VS. GENERATIVE UI

A VISUAL TEASER

TRADITIONAL APP

- Generic, crowded menus.
- Deep navigation trees.
- Endless filtering to find what you need.

GENERATIVE UI

- Instant, intent-matching solutions.
- Frictionless conversation.
- Actionable, native widgets loaded instantly.

Hi, I’m Balaji Venkatraman

- Lead 1 Software Engineer at UST Global
- 7+ years of experience in Flutter and React.

BEYOND CHATBOTS

GenUI = AI intent + UI Widgets

Power of Chatbots

Chatbots provide instant, context-aware information at our fingertips and dramatically reduce search friction.

Why Widgets Still matter

Humans complete tasks through interactive UI, so widgets remain essential for clarity, control, and action.

Magic of GenUI

GenUI combines AI intent understanding with structured widgets to generate dynamic, actionable interfaces in real time.

FROM TEXT TO WIDGETS

What is GenUI & Why Do We Need It?

The Definition

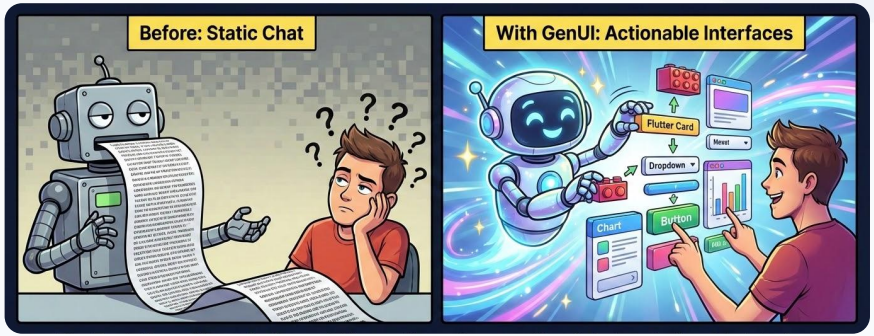
Generative UI (GenUI) is an architectural paradigm where AI dynamically assembles native user interface components in real-time based on user intent, shifting the AI's role from a text-based chatbot to a frontend orchestrator.

The Shift

Moves AI from being a simple text-based chatbot to a powerful frontend orchestrator.

The Result

Actionable, interactive interfaces built on the fly.



Contextual | Dynamic | Actionable

Composable | Adaptive | Reactive

THE BRIDGE

Meet the Flutter GenUI SDK

genui package is Flutter version of GenUI for building dynamic, conversational user interfaces powered by generative AI models.

genui allows you to create applications where the UI is not static or predefined, but is instead constructed by an AI in real-time based on a conversation with the user. This enables highly flexible, context-aware, and interactive user experiences.

https://pub.dev/packages/genui

Experimental
Version

9.3K
Downloads

76
Issues

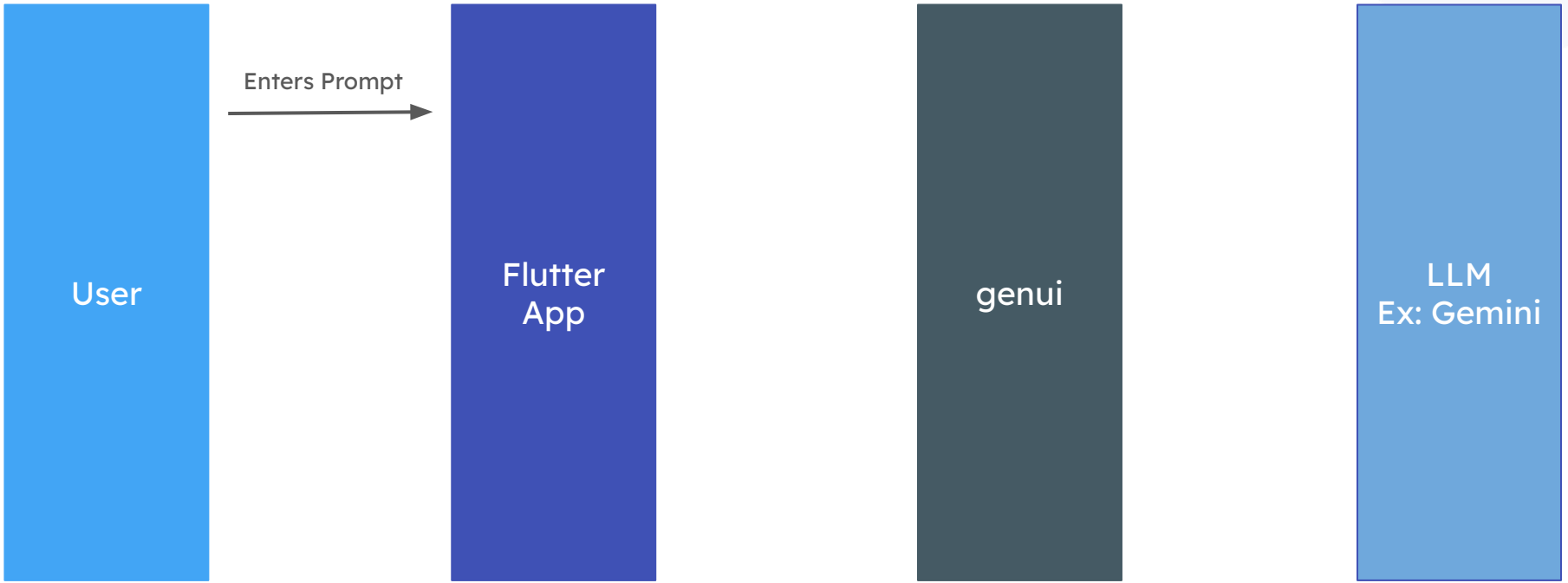
10
Open PRs

45 days ago
Last Shipped

160
Pub Points

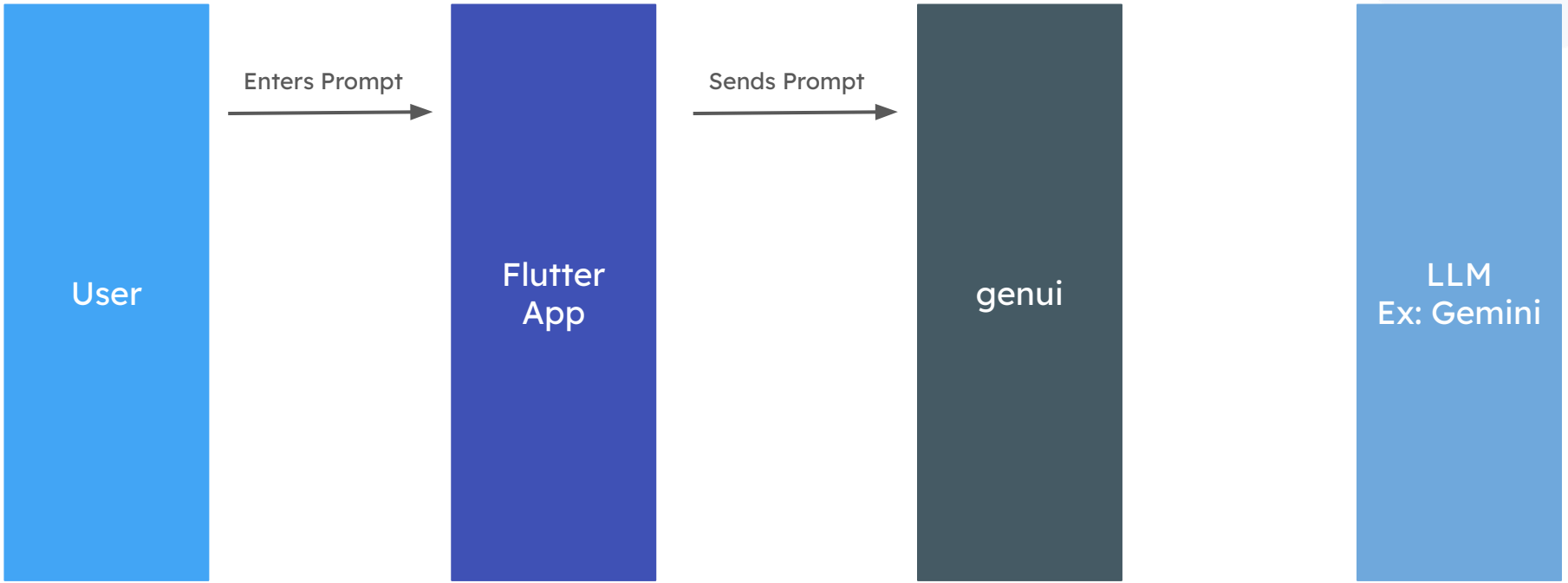
BEHIND THE SCENES

From Prompt to Pixels



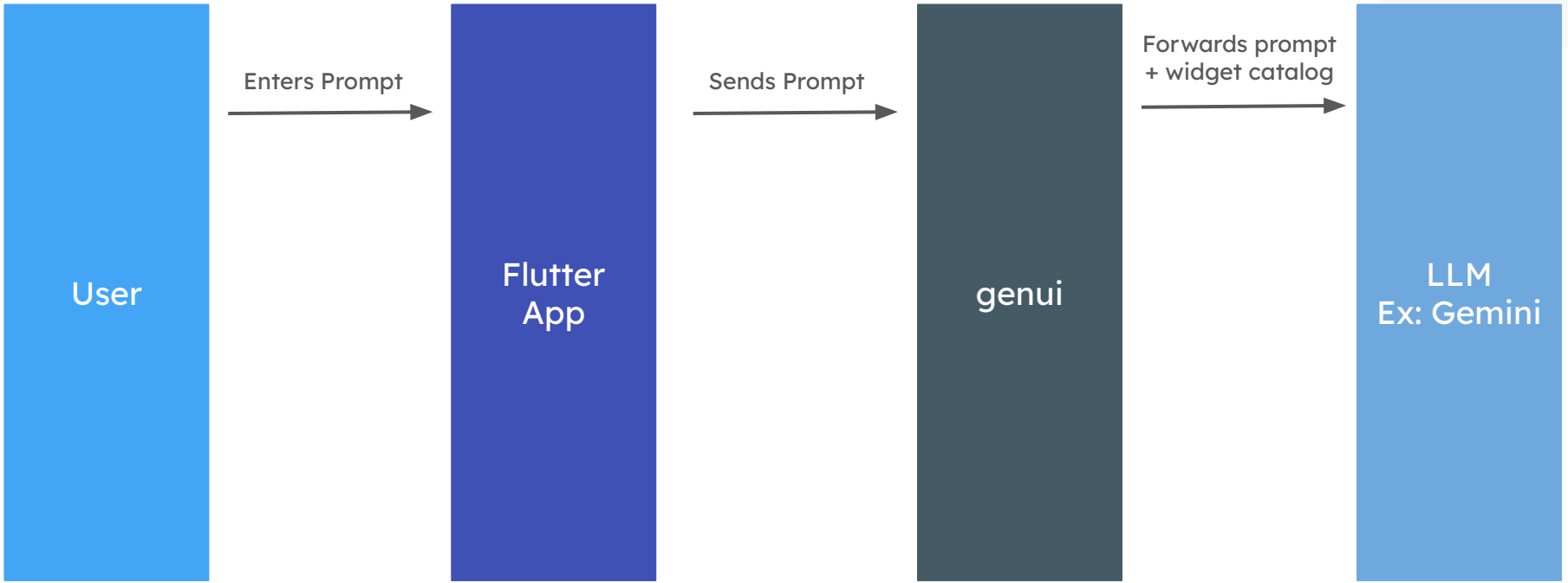
BEHIND THE SCENES

From Prompt to Pixels



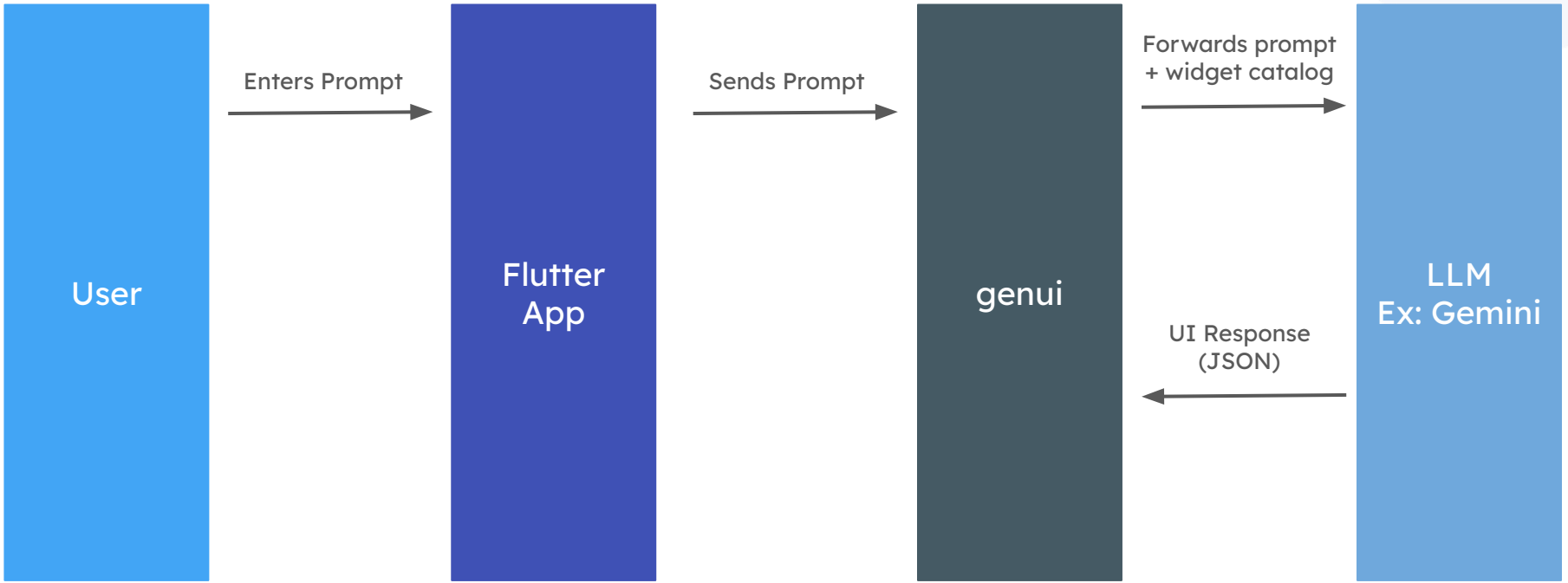
BEHIND THE SCENES

From Prompt to Pixels



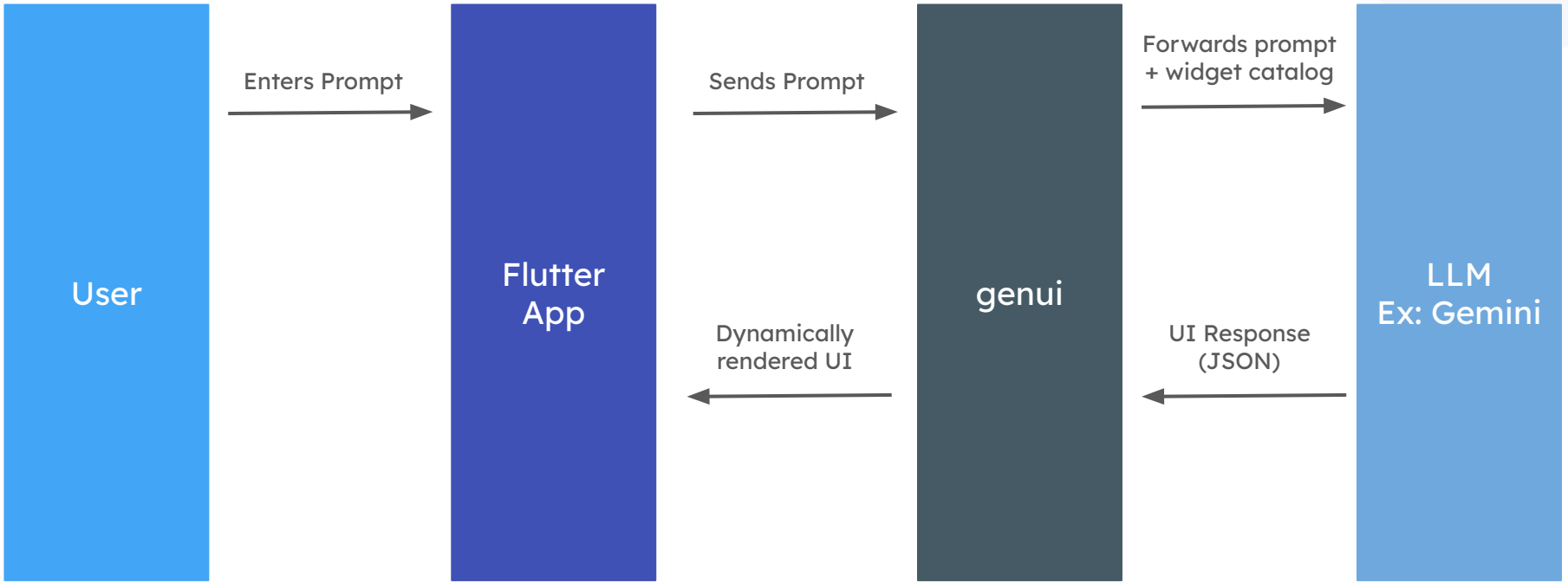
BEHIND THE SCENES

From Prompt to Pixels



BEHIND THE SCENES

From Prompt to Pixels



CORE PILLARS OF THE GENUI SDK

1. GenUiConversation

ORCHESTRATOR: Manages conversation history and the overall interaction cycle.

2. ContentGenerator

MODEL INTERFACE: Connects to the LLM for text and dynamic UI generation.

3. A2uiMessageProcessor

COMMAND CENTER: Handles AI messages to update DataModel and UI state.

4. DataModel

DYNAMIC STATE: A centralized, observable store for all real-time UI data.

5. Catalog

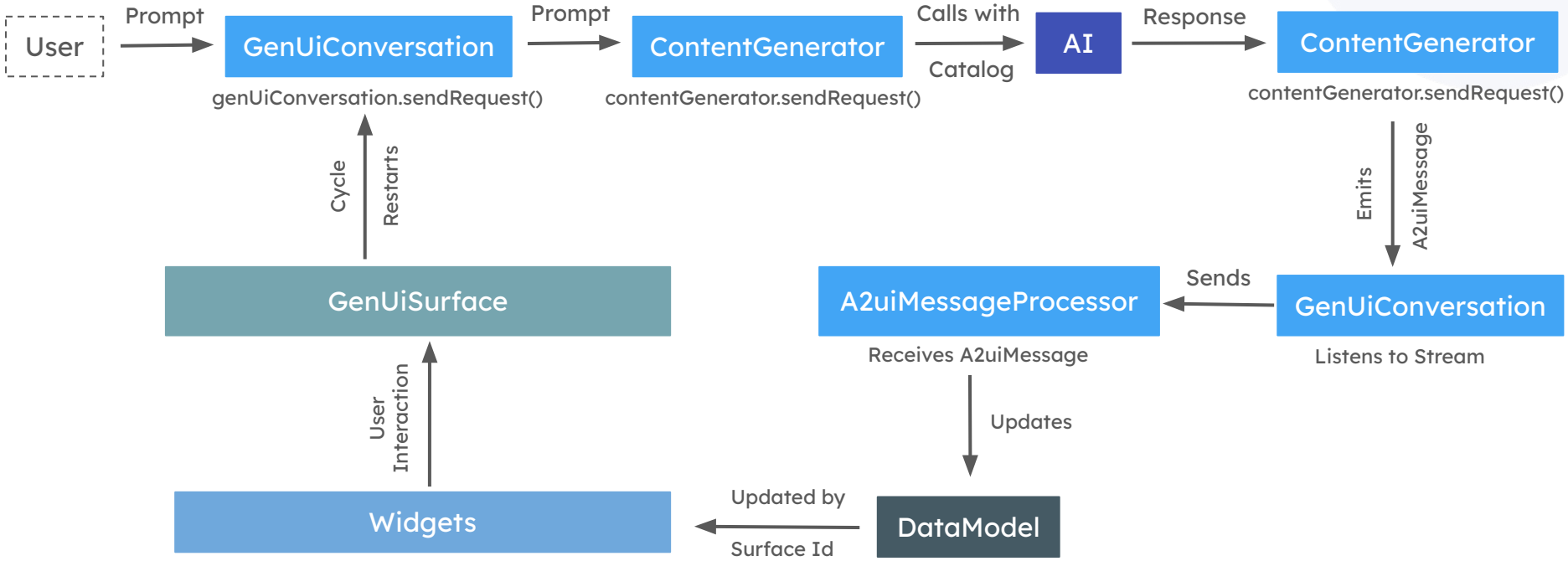
WIDGET TOOLKIT: Defines available native widgets the AI can render.

6. Surface

DYNAMIC RENDER ZONE: Flexible containers where AI paints native widgets on-the-fly.

SYSTEM ARCHITECTURE

How Components of GenUI works internally?



INTO THE CODE

Building the GenUI Pipeline

1. Package Installation

Add **genui** package and required agent provider packages if needed

```
genui: ^0.7.0
genui_firebase_ai: ^0.7.0
# genui_google_generative_ai
# genui_a2ui
```

pubscpec.yaml

INTO THE CODE

Building the GenUI Pipeline

2. Configure Catalog widget

Enable custom widgets with widget name, custom schema for properties(using json_schema_builder), and a builder function

```dart
final Schema _petTopicAdviceSchema = S.object(
  properties:
    'title': S.string(),
    'topic': S.string(),
    'summary': S.string(),
    'bullets': S.list(items: S.string()),
    'dos': S.list(items: S.string()),
    'donts': S.list(items: S.string()),
    'sources': S.list(items: S.string()),
  },
  required:  'title', 'topic', 'summary'],
);         [
```

```dart
final CatalogItem petTopicAdvice = CatalogItem(
  name: 'PetTopicAdvice',
  dataSchema: _petTopicAdviceSchema,
  widgetBuilder:  itemContext) {
    final data =(itemContext.data as JsonMap;
    return _PetTopicAdviceBody(
      title: _str(data, 'title', ''),
      topic: _str(data, 'topic', 'general'),
      summary: _str(data, 'summary', ''),
      bullets: _stringList(data['bullets']),
      dos: _stringList(data['dos']),
      donts: _stringList(data['donts']),
      sources: _stringList(data['sources']),
    );
  },
);
```

INTO THE CODE

Building the GenUI Pipeline

3. Configure Agent Provider

Configure agent providers. Agent providers helps to connect to LLMs. Choose from
1. Google Gemini AI,
2, Firebase AI Logic
3. GenUI A2UI

```
final contentGenerator = FirebaseAiContentGenerator(
      catalog: catalog,
      systemInstruction: _foodSystemInstruction,
      additionalTools:
        createPetFoodHardcodedSearchTool(),
      ],
      modelCreator:
          ({
            required FirebaseAiContentGenerator configuration
,
            firebase_ai.Content? systemInstruction,
            List<firebase_ai.Tool>? tools,
            firebase_ai.ToolConfig? toolConfig,
          }) {
            return GeminiGenerativeModel(
              firebaseAI.generativeModel(
                model: FirebaseAiConfig.generativeModel,
                systemInstruction: systemInstruction,
                tools: tools,
                toolConfig: toolConfig,
              ),
            );
          },
    );
```

INTO THE CODE

Building the GenUI Pipeline

4. Create A2uiMessageProcessor

Import **A2uiMessageProcessor** and attach the widget catalog using copyWith function or directly

```
final catalog = foodCatalog;
   final firebaseAI = ref.read(firebaseAIProvider);

   final processor = A2uiMessageProcessor(catalogs:  catalog
]);                                                  [
```

INTO THE CODE

Building the GenUI Pipeline

5. Create Conversation

Create the genui conversation by attaching the processor and content generator

```
_conversation = GenUiConversation(
      a2uiMessageProcessor: processor,
      contentGenerator: contentGenerator,
      onSurfaceAdded:  e) {
        setState(() {(
          if  !_surfaceIds.contains(e.surfaceId)) {
            (surfaceIds.add(e.surfaceId);
          }
        });
      },
      onSurfaceDeleted:  e) {
        setState(() {  (
          _surfaceIds.remove(e.surfaceId);
        });
      },
    );
```

INTO THE CODE

Building the GenUI Pipeline

6. Surface Management

Maintain a surface list to keep track of widgets created by genui sdk. The Surface list will contain ids of widgets which are rendered using GenUiSurface

Surface ids can be updated using onSurfaceAdded and onSurfaceDeleted functions

```
final List<String> _surfaceIds =
.....                              [];

_conversation = GenUiConversation(
  a2uiMessageProcessor: processor,
  contentGenerator: contentGenerato
  onSurfaceAdded: (e) {
    setState(() {
      if  !_surfaceIds.contains(e.surfaceId)) {
        (surfaceIds.add(e.surfaceId);

}
})},
  onSurfaceDeleted: (e) {
    setState(() {
      _surfaceId  remove(e.surfaceId);
s.
})},
);
```

INTO THE CODE

Building the GenUI Pipeline

7. Tool configuration

Additional tools for tool calls can be configured within **additionalTools** properties of Agent provider

Additional tools enable google search, maps search or any custom logic

A custom tool can be created using DynamicTool class containing name, description, input parameters, invoke function

```
DynamicAiTool<JsonMap> createToysFetchTool(
FirebaseFirestore firestore) {
  return DynamicAiTool<JsonMap>(
    name: 'fetch_toys_from_firestore',
    description:

'REQUIRED before showing shop-able toys in the UI. Rea
ds the live '
,
    parameters: dsb.S.object(
      properties:
        'search_query': dsb.S.string(
          description:

'Filter hint, e.g. "teething", "chew", "ball". Use emp
ty string to fetch all.'
,
        ),
      },
    ),
    invokeFunctio : (args) async
n     final q = args['search_query']?.toString().trim
() ?? '';
      try
      } catch (e, st)
        appLog.warning(
'fetch_toys_from_firestore failed: $e\n$st');
        return
          'toys': <Object>[],
          'error': e.toString(),
        };

}   },
  );
}
```

INTO THE CODE

Building the GenUI Pipeline

8. Sending Request

Send the request to LLM along with user prompt and catalog data using genui conversation object

```
void _onPromptSend(String text) {
    if  _conversation.contentGenerator.isProcessing.
value)(return;
    final profile = ref.read(petProfileProvider);
    unawaited(
      _conversation.sendRequest(
        UserMessage.text(foodTabMessageWithProfile(
profile, text)),
      ),
    );
  }
```

INTO THE CODE

Building the GenUI Pipeline

10. Displaying results

The dynamic rendered widgets can be displayed using GenUiSurface along with its surface ids

```
Expanded(
              child: ListView.builder(
                padding: const EdgeInsets.fromLTRB(16
, 8, 16, 16),
                itemCount: _surfaceIds.length,
                itemBuilder:  context, index) {
                  final id =(_surfaceIds[index];
                  return Padding(
                    padding: const EdgeInsets.only(
bottom: 12),
                    child: GenUiSurface(
                      host: _conversation.host,
                      surfaceId: id,
                    ),
                  );
                },
              ),
            ),
```

DEMO

THE REALITY CHECK

Current Limitations & Challenges

Still in Beta

Generative UI (GenUI) is an architectural paradigm where AI dynamically assembles native user interface components in real-time based on user intent, shifting the AI's role from a text-based chatbot to a frontend orchestrator.

LLM Hallucinations

Moves AI from being a simple text-based chatbot to a powerful frontend orchestrator.

Tedious Tooling

Actionable, interactive interfaces built on the fly. (Visual: Your comic-style image of the robot assembling Flutter blocks)

Limited custom catalog

Generative UI (GenUI) is an architectural paradigm where AI dynamically assembles native user interface components in real-time based on user intent, shifting the AI's role from a text-based chatbot to a frontend orchestrator.

Complex Debugging

Tracing a UI bug is harder when the "code" generating the UI is actually a natural language prompt.

Latency & Cost

Generating UI via LLM takes time. Managing loading states is critical to prevent the app from feeling slow.

Ask your Questions

Follow for more updates

Thank you!